

Clase 5 – Arquitecturas Avanzadas en Deep Learning para Series de Tiempo

Oscar Rodríguez

Advanced Time Series Analysis

Octubre 2025

Motivación

- RNN clásicas sufren *desvanecimiento/explosión de gradientes* con secuencias largas; el entrenamiento es estrictamente secuencial.
- Arquitecturas modernas: **LSTM, GRU, TCN, Transformers**.
- Objetivos de la clase:
 - Entender la **matemática** / expansiones cerradas de cada arquitectura.
 - Claridad sobre **complejidad** y **paralelismo**.
 - Buenas prácticas para forecasting univariado y multivariado.

Explicación intuitiva

- **Problema de RNN clásicas:** como una persona que solo recuerda lo último que escuchó, olvidando lo anterior cuando la historia es larga.
- **LSTM:** añade una libreta de memoria con pestañas: puedes borrar (olvido), anotar (entrada) o mostrar (salida).
- **GRU:** es como la LSTM pero más compacta: solo tiene dos botones, uno decide si actualizas o mantienes, otro si usas lo viejo al proponer lo nuevo.
- **TCN:** en lugar de leer palabra por palabra, miras varios pasos atrás a la vez con “ventanas” de distinto tamaño (dilatación), y puedes hacerlo en paralelo.
- **Transformers:** cada paso pregunta directamente al resto “¿a quién debo prestar atención?” y recibe una mezcla de respuestas, sin pasar uno por uno.

Mapa de la clase

1 LSTM

2 GRU

3 TCN

4 Transformers

5 Cierre

LSTM – Definición y motivación

Las **Long Short-Term Memory (LSTM)** son una extensión de las RNN diseñadas para capturar dependencias a largo plazo. Mientras que las RNN clásicas tienden a sufrir el problema del *desvanecimiento del gradiente*, las LSTM introducen un mecanismo de **memoria controlada** mediante **puertas** (*gates*) que regulan el flujo de información.

- **Entrada:** x_i (vector de características en el tiempo i)
- **Salida:** $\hat{y}_i = \sigma(W_y h_i + b_y)$
- **Memoria interna:** c_i (estado de celda) y h_i (estado oculto)

A diferencia de la RNN:

$$h_i = f(W_h h_{i-1} + W_x x_i + b_h),$$

la LSTM introduce un sistema de puertas que modulan el ingreso, mantenimiento y salida de la información en el tiempo.

LSTM – Puertas

Cada celda LSTM contiene tres puertas principales y un estado de celda:

- **Puerta de olvido:** controla cuánto del estado anterior se conserva.
- **Puerta de entrada:** regula cuánta información nueva se agrega.
- **Puerta de salida:** decide cuánta información de la celda se expone al siguiente paso.

Comportamiento:

- El **estado de celda** c_t fluye casi linealmente a lo largo del tiempo:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Esta estructura permite **recordar información por intervalos largos** sin que el gradiente se desvanezca (por eso se denominan *Long Short-Term Memory*).

- El **estado oculto** h_t se usa para:
 - Producir la salida inmediata: $\hat{y}_t = W_y h_t + b_y$
 - Retroalimentar la celda en el siguiente paso temporal (h_{t-1}).

LSTM – Interpretación en series de tiempo

En éste contexto, los estados internos representan información acumulada:

- x_t : valor o vector observado en el tiempo t
- h_t : **resumen no lineal del pasado cercano** (información de corto plazo)
- c_t : **almacén de información relevante del pasado lejano** (memoria de largo plazo)

Por tanto, cuando el modelo desea predecir el siguiente valor:

$$\hat{x}_{t+1} = f(x_1, x_2, \dots, x_t)$$

utiliza principalmente el estado oculto h_t , el cual contiene una representación del pasado filtrada y consolidada a través del estado de celda c_t .

Interpretación práctica:

- c_t actúa como una “memoria de largo plazo” que almacena información relevante a lo largo de toda la secuencia.
- h_t es la “salida visible” del modelo en cada paso, reflejando lo que la red “piensa” en el instante actual.

LSTM – Estructura

Un LSTM tiene la siguiente estructura explícita:

$$f_i = \sigma(W_f x_i + U_f h_{i-1} + b_f) \quad (\text{Forget gate})$$

$$i_i = \sigma(W_i x_i + U_i h_{i-1} + b_i) \quad (\text{Input gate})$$

$$\tilde{c}_i = \tanh(W_c x_i + U_c h_{i-1} + b_c) \quad (\text{Candidato de memoria})$$

$$c_i = f_i \odot c_{i-1} + i_i \odot \tilde{c}_i \quad (\text{Actualización del estado de celda})$$

$$o_i = \sigma(W_o x_i + U_o h_{i-1} + b_o) \quad (\text{Output gate})$$

$$h_i = o_i \odot \tanh(c_i) \quad (\text{Estado oculto final})$$

donde $\odot$ representa el producto elemento a elemento (Hadamard).

LSTM – Deducción paso a paso

Suponiendo $h_0 = 0$ y $c_0 = 0$, se calculan los estados para $i = 1, \dots, N$:

$$f_1 = \sigma(W_f x_1 + U_f h_0 + b_f),$$

$$i_1 = \sigma(W_i x_1 + U_i h_0 + b_i),$$

$$\tilde{c}_1 = \tanh(W_c x_1 + U_c h_0 + b_c),$$

$$c_1 = f_1 \odot c_0 + i_1 \odot \tilde{c}_1,$$

$$o_1 = \sigma(W_o x_1 + U_o h_0 + b_o),$$

$$h_1 = o_1 \odot \tanh(c_1),$$

$$\hat{y}_1 = \sigma(W_y h_1 + b_y).$$

LSTM – Deducción paso a paso

De forma recursiva:

$$f_i = \sigma(W_f x_i + U_f h_{i-1} + b_f), \quad c_i = f_i \odot c_{i-1} + i_i \odot \tilde{c}_i, \quad h_i = o_i \odot \tanh(c_i),$$

Ejercicio práctico: Si $x_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}^q$ y la capa LSTM tiene a unidades, determina las dimensiones de cada matriz $W_f, U_f, W_i, U_i, \dots$

LSTM – Representación matricial

Podemos representar la propagación temporal en forma matricial:

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_N \end{bmatrix}, \quad C = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_N \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \quad \hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}.$$

Entonces, de forma compacta:

$$\begin{aligned} F &= \sigma(W_f X + U_f H_{-1} + b_f), & I &= \sigma(W_i X + U_i H_{-1} + b_i), \\ \tilde{C} &= \tanh(W_c X + U_c H_{-1} + b_c), & C &= F \odot C_{-1} + I \odot \tilde{C}, \\ O &= \sigma(W_o X + U_o H_{-1} + b_o), & H &= O \odot \tanh(C), \\ \hat{Y} &= \sigma(W_y H + b_y). \end{aligned}$$

Nota: La LSTM mantiene información de largo plazo mediante el estado c_i , lo cual permite modelar secuencias complejas y dependencias temporales extendidas.

LSTM – Entrenamiento y función de pérdida

El objetivo de una LSTM en una tarea de predicción de series de tiempo es aproximar:

$$\hat{x}_{t+1} = f(x_1, x_2, \dots, x_t)$$

Minimizando una función de pérdida como el **error cuadrático medio (MSE)**:

$$L = \frac{1}{T} \sum_{t=1}^T (x_t - \hat{x}_t)^2$$

El entrenamiento se realiza mediante **backpropagation through time (BPTT)**, actualizando todos los pesos:

$$W = \{W_f, W_i, W_c, W_o, U_f, U_i, U_c, U_o, W_y\}.$$

Intuición:

- Si $f_t \approx 1$: la celda conserva memoria.
- Si $f_t \approx 0$: olvida la información anterior.
- Si $i_t \approx 1$: incorpora nueva información.
- Si $o_t \approx 1$: expone el contenido de la memoria.

Mapa de la clase

1 LSTM

2 GRU

3 TCN

4 Transformers

5 Cierre

GRU – Definición y motivación

Las **Gated Recurrent Units (GRU)** son una variante simplificada de las LSTM, introducidas por Cho et al. (2014), diseñadas para capturar dependencias de largo plazo con una arquitectura más compacta. El objetivo es mantener la capacidad de aprendizaje temporal reduciendo la complejidad computacional.

- **Entrada:** x_i (vector de características en el tiempo i)
- **Salida:** $\hat{y}_i = \sigma(W_y h_i + b_y)$
- **Memoria interna:** h_i (único estado, combina corto y largo plazo)

A diferencia de la LSTM, la GRU no tiene un estado de celda c_i separado. En su lugar, utiliza dos **puertas de control**:

- La **puerta de actualización** z_t : regula cuánto del pasado se mantiene.
- La **puerta de reinicio** r_t : controla cuánto del pasado se olvida.

GRU – Puertas y comportamiento

Cada celda GRU contiene dos puertas principales que controlan el flujo de información:

- **Puerta de actualización (z_t):** decide cuánto del estado anterior h_{t-1} conservar.
- **Puerta de reinicio (r_t):** decide qué información previa debe ser olvidada al generar el nuevo estado.

Comportamiento:

- La **puerta de actualización** combina la memoria pasada con la nueva información:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

Si $z_t \approx 0$, el modelo mantiene la información anterior; si $z_t \approx 1$, incorpora información nueva.

- La **puerta de reinicio** modula el acceso al pasado:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h).$$

Intuición: El estado oculto h_t combina la función de memoria y salida, simplificando la arquitectura respecto a las LSTM.

GRU – Interpretación en series de tiempo

En el contexto de una **serie de tiempo**, el estado interno de la GRU representa la información temporal acumulada hasta el instante t :

- x_t : valor o vector observado en el tiempo t .
- h_t : **memoria comprimida del sistema**, que combina información del pasado cercano y lejano.

Por tanto, cuando el modelo desea predecir el siguiente valor:

$$\hat{x}_{t+1} = f(x_1, x_2, \dots, x_t),$$

utiliza el estado oculto h_t , que encapsula la información relevante de la secuencia previa gracias al control dinámico de las puertas z_t y r_t .

Interpretación práctica:

- z_t regula la persistencia de la memoria.
- r_t ajusta la influencia del pasado.
- h_t actúa como resumen contextual del sistema.

GRU – Estructura

Un GRU tiene la siguiente estructura explícita:

$$\begin{aligned} z_i &= \sigma(W_z x_i + U_z h_{i-1} + b_z) && \text{(Puerta de actualización)} \\ r_i &= \sigma(W_r x_i + U_r h_{i-1} + b_r) && \text{(Puerta de reinicio)} \\ \tilde{h}_i &= \tanh(W_h x_i + U_h(r_i \odot h_{i-1}) + b_h) && \text{(Estado candidato)} \\ h_i &= (1 - z_i) \odot h_{i-1} + z_i \odot \tilde{h}_i && \text{(Nuevo estado oculto final)} \end{aligned}$$

donde σ es la función sigmoide y $\odot$ representa el producto elemento a elemento (Hadamard).

GRU – Deducción paso a paso

Suponiendo $h_0 = 0$, se calculan los estados para $i = 1, \dots, N$:

$$z_1 = \sigma(W_z x_1 + U_z h_0 + b_z),$$

$$r_1 = \sigma(W_r x_1 + U_r h_0 + b_r),$$

$$\tilde{h}_1 = \tanh(W_h x_1 + U_h(r_1 \odot h_0) + b_h),$$

$$h_1 = (1 - z_1) \odot h_0 + z_1 \odot \tilde{h}_1,$$

$$\hat{y}_1 = \sigma(W_y h_1 + b_y).$$

GRU – Deducción paso a paso

De forma recursiva:

$$\begin{aligned} z_i &= \sigma(W_z x_i + U_z h_{i-1} + b_z), & r_i &= \sigma(W_r x_i + U_r h_{i-1} + b_r), \\ \tilde{h}_i &= \tanh(W_h x_i + U_h(r_i \odot h_{i-1}) + b_h), & h_i &= (1 - z_i) \odot h_{i-1} + z_i \odot \tilde{h}_i, \\ \hat{y}_i &= \sigma(W_y h_i + b_y). \end{aligned}$$

Ejercicio práctico: Si $x_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}^q$ y la capa GRU tiene a unidades, determina las dimensiones de las matrices $W_z, U_z, W_r, U_r, W_h, U_h$.

GRU – Representación matricial

Podemos representar la propagación temporal en forma matricial:

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_N \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \quad \hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}.$$

Entonces, de forma compacta:

$$\begin{aligned} Z &= \sigma(W_z X + U_z H_{-1} + b_z), & R &= \sigma(W_r X + U_r H_{-1} + b_r), \\ \tilde{H} &= \tanh(W_h X + U_h (R \odot H_{-1}) + b_h), & H &= (1 - Z) \odot H_{-1} + Z \odot \tilde{H}, \\ \hat{Y} &= \sigma(W_y H + b_y). \end{aligned}$$

Nota: La GRU combina en un único estado h_t tanto la memoria de corto como la de largo plazo, simplificando su arquitectura sin perder capacidad de representación temporal.

GRU – Entrenamiento y función de pérdida

El objetivo de una GRU en una tarea de predicción de series de tiempo es aproximar:

$$\hat{x}_{t+1} = f(x_1, x_2, \dots, x_t).$$

Minimizando una función de pérdida como el **error cuadrático medio (MSE)**:

$$L = \frac{1}{T} \sum_{t=1}^T (x_t - \hat{x}_t)^2.$$

El entrenamiento se realiza mediante BPTT, actualizando los pesos:

$$W = \{W_z, W_r, W_h, U_z, U_r, U_h, W_y\}.$$

Intuición:

- Si $z_t \approx 0$: la red mantiene la memoria anterior.
- Si $z_t \approx 1$: la red adopta nueva información.
- Si $r_t \approx 0$: se ignora el contexto pasado.

Conclusión: Las GRU son más eficientes que las LSTM, manteniendo un desempeño competitivo en la mayoría de tareas secuenciales.

Mapa de la clase

1 LSTM

2 GRU

3 TCN

4 Transformers

5 Cierre

TCN – Definición y motivación

Las **Temporal Convolutional Networks (TCN)** son una alternativa moderna a las redes recurrentes (RNN, LSTM, GRU) para el modelado de secuencias temporales. El principio fundamental es reemplazar la recurrencia por **convoluciones causales dilatadas**, lo que permite capturar dependencias de largo plazo de manera eficiente y estable.

- **Entrada:** x_i (vector de características en el tiempo i)
- **Salida:** $\hat{y}_i = f(x_1, x_2, \dots, x_i)$
- **Mecanismo:** Convoluciones 1D causales con dilatación creciente

Ventajas principales:

- Procesamiento **paralelo**, sin recurrencia.
- Gran **receptive field** (campo receptivo) con pocas capas.
- Estabilidad en el gradiente y entrenamiento más rápido.

TCN – Arquitectura y comportamiento

Las TCN utilizan **convoluciones causales dilatadas** para modelar dependencias temporales sin recurrencia:

$$y_t = \sum_{k=0}^{K-1} w_k x_{t-d \cdot k},$$

donde:

- K : tamaño del kernel (ventana de tiempo).
- d : factor de dilatación.
- w_k : pesos del filtro convolucional.

Comportamiento: La **causalidad** asegura que y_t dependa solo de entradas anteriores o actuales ($x_{\leq t}$), y la **dilatación** expande el rango de tiempo considerado sin aumentar el número de parámetros.

Ejemplo de dilatación:

Para $d = 1 : [x_t, x_{t-1}, x_{t-2}]$, para $d = 2 : [x_t, x_{t-2}, x_{t-4}]$.

TCN – Interpretación en series de tiempo

En el contexto de una **serie de tiempo**, las TCN actúan como un conjunto de **filtros jerárquicos temporales**:

- x_t : valor o vector de entrada en el tiempo t .
- h_t : características extraídas por convolución (resumen del contexto pasado).
- $\hat{x}_{t+1}$: predicción basada en la ventana causal del pasado.

Ventaja clave:

- Las dependencias de largo plazo se modelan mediante la **dilatación exponencial**, sin necesidad de estados recurrentes ni memoria explícita.

Analogía:

- LSTM: usa memoria recurrente,
- TCN: usa memoria convolucional (ventanas dilatadas).

TCN – Estructura

Una TCN está compuesta por varios **bloques residuales** con convoluciones causales dilatadas:

$$h_t^{(l)} = \text{ReLU}\left(W^{(l)} *_{d_l} h_t^{(l-1)} + b^{(l)}\right),$$

donde:

- $*_{d_l}$ indica una convolución dilatada con factor d_l .
- $h_t^{(l)}$ es la salida de la capa l .
- $d_l = 2^l$ suele crecer exponencialmente con la profundidad.

Cada bloque incluye una **conexión residual** que mejora la estabilidad del gradiente:

$$h_t^{(l)} = h_t^{(l-1)} + \text{Dropout}\left(\text{ReLU}\left(\text{Conv1D}_{d_l}(h_t^{(l-1)})\right)\right).$$

La salida final se obtiene mediante:

$$\hat{y}_t = W_y h_t^{(L)} + b_y.$$

TCN – Deducción paso a paso

Consideremos una TCN con dos capas convolucionales y factores de dilatación $d_1 = 1$ y $d_2 = 2$:

$$\begin{aligned}h_t^{(1)} &= \text{ReLU}\left(w_0^{(1)}x_t + w_1^{(1)}x_{t-1} + w_2^{(1)}x_{t-2} + b^{(1)}\right), \\h_t^{(2)} &= \text{ReLU}\left(w_0^{(2)}h_t^{(1)} + w_1^{(2)}h_{t-2}^{(1)} + w_2^{(2)}h_{t-4}^{(1)} + b^{(2)}\right), \\ \hat{y}_t &= W_y h_t^{(2)} + b_y.\end{aligned}$$

El **campo receptivo** (*receptive field*) crece exponencialmente con el número de capas y la dilatación:

$$R = 1 + (K - 1) \sum_{l=0}^{L-1} 2^l.$$

De esta forma, con pocas capas, la red puede “ver” una gran porción del pasado.

TCN – Representación matricial

El proceso convolucional puede representarse de forma matricial (tipo Toeplitz):

$$H = X *_d W + b,$$

donde:

- X es una matriz temporal con desplazamientos causales.
- W son los filtros convolucionales.
- d es el factor de dilatación (espaciado entre elementos).

Ejemplo: Para $d = 2$ y $K = 3$:

$$\begin{bmatrix} x_t & x_{t-2} & x_{t-4} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = y_t.$$

Esta formulación muestra que las TCN pueden implementarse como Conv1D con **padding causal**, garantizando la no dependencia futura.

TCN – Entrenamiento y función de pérdida

El objetivo de una TCN en series de tiempo es aproximar:

$$\hat{x}_{t+1} = f(x_t, x_{t-1}, \dots, x_{t-R}),$$

donde R representa el tamaño efectivo del *receptive field*.

La función de pérdida típica es el **error cuadrático medio (MSE)**:

$$L = \frac{1}{T} \sum_{t=1}^T (x_t - \hat{x}_t)^2.$$

Entrenamiento:

- Se realiza mediante **backpropagation estándar**, sin recurrencia (no se requiere BPTT).
- Los gradientes se calculan capa por capa, como en una CNN.

Ventajas prácticas:

- Evita el problema del gradiente desvanecido/explosivo.
- Permite entrenamiento paralelo en todos los pasos temporales.
- Control explícito del contexto mediante la dilatación.

Mapa de la clase

1 LSTM

2 GRU

3 TCN

4 Transformers

5 Cierre

Transformers – Definición y motivación

Los **Transformers** (Vaswani et al., 2017) representan un cambio fundamental en el modelado de secuencias: reemplazan la recurrencia y las convoluciones por un mecanismo de **atención** que permite procesar todos los elementos de la secuencia en paralelo.

- **Entrada:** secuencia $\{x_1, x_2, \dots, x_N\}$
- **Salida:** $\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N\}$
- **Mecanismo:** atención (self-attention) + proyecciones lineales

Ventajas principales:

- Procesamiento totalmente **paralelo**.
- Captura de **dependencias globales** sin recurrencia.
- Mejor escalabilidad y estabilidad en el entrenamiento.

Transformers – Arquitectura general

Un Transformer se compone de dos bloques principales:

- **Codificador (Encoder):** transforma la secuencia de entrada en representaciones contextuales.
- **Decodificador (Decoder):** genera la salida paso a paso, utilizando la información del encoder y del contexto previo.

Estructura del encoder:

- Multi-Head Self-Attention
- Normalización de capa (LayerNorm)
- Feed-Forward Network (FFN)
- Conexiones residuales

$$h^{(l)} = \text{LayerNorm}\left(h^{(l-1)} + \text{MultiHeadAttn}(h^{(l-1)})\right)$$

$$h^{(l)} = \text{LayerNorm}\left(h^{(l)} + \text{FFN}(h^{(l)})\right)$$

Transformers – Atención y comportamiento

El **mecanismo de atención** permite que cada posición de la secuencia “observe” a las demás posiciones, asignando pesos de importancia a cada una. Para una secuencia de entrada $X \in \mathbb{R}^{N \times d_{model}}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

donde:

- $Q = XW_Q$: consultas (queries)
- $K = XW_K$: claves (keys)
- $V = XW_V$: valores (values)
- d_k : dimensión de las claves

Interpretación:

- Cada elemento “atiende” a los demás para construir una representación contextual.
- La atención mide la **relevancia** entre elementos mediante productos punto.

Transformers – Multi-Head Attention

Para capturar múltiples relaciones en paralelo, se usa el **multi-head attention**:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

donde:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Ventajas:

- Cada cabeza aprende un tipo diferente de dependencia.
- Permite capturar contextos a distintas escalas o subespacios.

Dimensiones típicas:

- $d_{\text{model}} = 512$
- $h = 8$ cabezas
- $d_k = d_v = 64$

Transformers – Codificación posicional

A diferencia de las RNN, los Transformers no tienen noción inherente de orden temporal. Por ello, se agregan **positional encodings** a las entradas:

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

La entrada final al modelo es:

$$X' = X + \text{PE}$$

Interpretación:

- Permite distinguir el orden de los elementos.
- Añade información periódica sobre la posición en la secuencia.

Transformers – Flujo paso a paso

Codificador:

$$Z_0 = X + \text{PE},$$

$$Z_l = \text{LayerNorm}(Z_{l-1} + \text{MultiHeadAttn}(Z_{l-1})),$$

$$H_l = \text{LayerNorm}(Z_l + \text{FFN}(Z_l)).$$

Decodificador:

$$Y_l = \text{MaskedMultiHeadAttn}(Y_{l-1}),$$

$$Y_l = \text{LayerNorm}(Y_l + \text{CrossAttn}(Y_l, H_L)),$$

$$\hat{Y} = \text{Softmax}(W_o Y_L + b_o).$$

Salida final:

$$\hat{y}_t = P(y_t | y_{<t}, X)$$

donde el decodificador se enmascara para evitar usar información futura.

Transformers – Representación matricial

El proceso de atención puede verse como una operación completamente matricial:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right), \quad H = AV.$$

Dimensiones:

$$Q, K, V \in \mathbb{R}^{N \times d_k}, \quad A \in \mathbb{R}^{N \times N}, \quad H \in \mathbb{R}^{N \times d_v}.$$

En el caso del **multi-head attention**:

$$H = \text{Concat}(H_1, \dots, H_h)W_O.$$

Interpretación: Cada fila de A representa la distribución de atención que un elemento dedica al resto de la secuencia.

Transformers – Entrenamiento y función de pérdida

El objetivo de un Transformer en series de tiempo o tareas de secuencia es aproximar:

$$\hat{x}_{t+1} = f(x_1, x_2, \dots, x_t).$$

La función de pérdida típica es la **entropía cruzada** para clasificación o el **error cuadrático medio (MSE)** para regresión:

$$L = -\frac{1}{T} \sum_{t=1}^T y_t \log(\hat{y}_t) \quad \text{o} \quad L = \frac{1}{T} \sum_{t=1}^T (x_t - \hat{x}_t)^2.$$

Ventajas prácticas:

- Escalabilidad a grandes secuencias y datasets.
- Captura de dependencias globales sin recurrencia.
- Entrenamiento altamente paralelizable.

Comparativa e intuiciones

Arquitectura	Ventajas	Limitaciones
LSTM	Ruta de memoria explícita; pesos de supervivencia	Más parámetros; costo secuencial
GRU	Menos parámetros; mezcla exponencial adaptativa	Menos control explícito que LSTM
TCN	Paralelizable; campo receptivo controlable	Dependencia acotada por R si no se aumenta profundidad/dilatación
Transformer	Dependencias globales; paralelizable	$O(T^2)$ memoria/tiempo; requiere datos y tuning

Mapa de la clase

1 LSTM

2 GRU

3 TCN

4 Transformers

5 Cierre

Bibliografía y recursos

Originales

- LSTM: Hochreiter, S. & Schmidhuber, J. (1997). *Long Short-Term Memory*. [PDF](#).
- GRU: Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for SMT*. [arXiv:1409.0473](#).
- TCN (visión): Lea, C. et al. (2017). *Temporal Convolutional Networks*. [CVPR 2017](#).
- TCN (secuencias): Bai, S., Kolter, J.Z., Koltun, V. (2018). *An Empirical Evaluation....* [arXiv:1803.01271](#).
- Transformers: Vaswani, A. et al. (2017). *Attention is All You Need*. [arXiv:1706.03762](#).

Recursos didácticos

- Towards Data Science: [Math behind GRU](#).
- GeeksforGeeks: [Intro LSTM](#).

Siguiente clase

- Vimos expansiones cerradas (LSTM/GRU), receptivos (TCN) y pesos de atención (Transformers).
- Próximos temas: **incertidumbre, interpretabilidad y control probabilístico** (*Inferencia Bayesiana y Deep Probabilístico*).